Lab Manual

AIC 380 – Artificial Neural Networks

(1st Draft Version)

CUI

Department of Computer Science
Islamabad Campus

Lab Contents:

This lab manual provides an understanding of implementing artificial neural networks in python. The topics include: implementation of perceptron; artificial neural networks; self-organizing maps; learning vector quanizations;; Convolutional Neural Networks; Recurrent Neural Networks; long-short term memory.

Student Outcomes (SO)

S.#	Description
2	Overview of Neural Networks: Definition, History, Biological Neurons, Models of Neurons, Artificial & Biological Neural Networks, and Neural Network Models
3	Single Layer Perceptrons: Least Mean Square Algorithm, Learning Curves, Learning Rates, Perceptron; Multilayer Perceptrons: XOR Problem, Backpropagation Algorithm, Heuristic for Improving the Backpropagation Algorithm; and Adaline & Madaline Networks.
4	Pattern Association: Hetroassociative Memory Neural Network, Autoassociative Neural Network, Iterative Autoassociative Net; Bidirectional Associative Memory (BAM) & Hopfield Memory: Architecture, Algorithm, Analysis, and Applications
5	Radial-Basis Function Networks: Interpolation, and Regularization & Learning strategies.
6	Kohonen Self-Organizing Maps: SOM Algorithm, Learning Vector Quantization; Adaptive Resonance Theory (ART); Spatiotemporal Networks (STNS); and Neocognitron.
7	Convolutional Neural Networks (CNN): Motivation, Convolutional layers, Additional layers, Residual Nets; Recurrent Neural Networks (RNN): Motivation, Sequential Processing, Stability, and Gated Nets (LSTM, GRU).

Intended Learning Outcomes

Sr.#	Description	Blooms Taxonomy Learning Level	SO
CLO -2	Apply appropriate Neural Networks to solve practical	*Applying*	2-4
CLO -3	Apply deep learning techniques for classification and recognition problems.	*Applying*	2-4
CLO-4	Implement appropriate Neural Networks to solve practical problems.	*Applying*	2-4

Lab Assessment Policy

The lab work done by the student is evaluated using Psycho-motor rubrics defined by the course instructor, viva-voce, project work/performance. Marks distribution is as follows:

Assignments	Lab Mid Term Exam	Lab Terminal Exam	Total
25	25	50	100

Note: Midterm and Final term exams must be computer based.

List of Labs

Lab 01

Objective:

The objective of this lab is to give students a recap of python basics important for understanding the implementation of artificial neural networks in python.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Implement a data type in python.
- Implement flow control statement.
- Implement functions and classes.
- Implement file IO.

Instructor Note:

As pre-lab activity, read relevant chapters from Madhavan, Samir. Mastering python for data science. Packt Publishing Ltd, 2015: https://github.com/AmandaZou/Data-Science-books-/blob/master/Mastering%20Python%20for%20Data%20Science.pdf

1) Useful Concepts

1. **Syntax:** Python has no mandatory statement termination characters and blocks are specified by indentation. Indent to begin a block, dedent to end one. Statements that expect an indentation level end in a colon (:). Comments start with the pound (#) sign and are single-line, multi-line strings are used for multi-line comments. Values are assigned (in fact, objects are bound to names) with the equals sign ("="), and equality testing is done using two equals signs ("=="). You can increment/decrement values using the += and -= operators respectively by the right-hand amount. This works on many datatypes, strings included. You can also use multiple variables on one line. For example:

```
>>> myvar = 3
>>> myvar += 2
>>> myvar
5
>>> myvar -= 1
>>> myvar
4
"""This is a multiline comment.
The following lines concatenate the two strings."""
>>> mystring = "Hello"
>>> mystring += " world."
>>> print(mystring)
Hello world.
# This swaps the variables in one line(!).
# It doesn't violate strong typing because values aren't
# actually being assigned, but new objects are bound to
# the old names.
>>> myvar, mystring = mystring, myvar
```

2. **Data types:** The data structures available in python are lists, tuples and dictionaries. Sets are available in the sets library (but are built-in in Python 2.5 and later). Lists are like one-dimensional arrays (but you can also have lists of other lists), dictionaries are associative arrays (a.k.a. hash tables) and tuples are immutable one-dimensional arrays (Python "arrays" can be of any type, so you can mix e.g. integers, strings, etc in lists/dictionaries/tuples). The index of the

first item in all array types is 0. Negative numbers count from the end towards the beginning, -1 is the last item. Variables can point to functions. The usage is as follows

```
>>> sample = [1, ["another", "list"], ("a", "tuple")]
>>> mylist = ["List item 1", 2, 3.14]
>>> mylist[0] = "List item 1 again" # We're changing the item.
>>> mylist[-1] = 3.21 # Here, we refer to the last item.
>>> mydict = {"Key 1": "Value 1", 2: 3, "pi": 3.14}
>>> mydict["pi"] = 3.15 # This is how you change dictionary values
>>> mytuple = (1, 2, 3)
>>> myfunction = len
>>> print(myfunction(mylist))
```

You can access array ranges using a colon (:). Leaving the start index empty assumes the first item, leaving the end index assumes the last item. Indexing is inclusive-exclusive, so specifying [2:10] will return items [2] (the third item, because of 0-indexing) to [9] (the tenth item), inclusive (8 items). Negative indexes count from the last item backwards (thus -1 is the last item) like so:

```
>>> mylist = ["List item 1", 2, 3.14]
>>> print(mylist[:])
['List item 1', 2, 3.1400000000000001]
>>> print(mylist[0:2])
['List item 1', 2]
>>> print(mylist[-3:-1])
['List item 1', 2]
>>> print(mylist[1:])
[2, 3.14]
# Adding a third parameter, "step" will have Python step in
# N item increments, rather than 1.
# E.g., this will return the first item, then go to the third and
# return that (so, items 0 and 2 in 0-indexing).
>>> print(mylist[::2])
['List item 1', 3.14]
```

3. **Strings:** Its strings can use either single or double quotation marks, and you can have quotation marks of one kind inside a string that uses the other kind (i.e. “He said ’hello’.” is valid). Multiline strings are enclosed in _triple double (or single) quotes_ (“””). Python strings are always Unicode, but there is another string type that is pure bytes. Those are called bytestrings and are represented with the b prefix, for example b'Hello \xce\xb1'. . To fill a string with values, you use the % (modulo) operator and a tuple. Each %s gets replaced with an item from the tuple, left to right, and you can also use dictionary substitutions, like so:

```
>>> print("Name: %s\
Number: %s\
String: %s" % (myclass.name, 3, 3 * "-"))
Name: Stavros
Number: 3
String: ---

strString = """This is
a multiline
string."""

# WARNING: Watch out for the trailing s in "%(key)s".
>>> print("This %(verb)s a %(noun)s." % {"noun": "test", "verb": "is"})
This is a test.

>>> name = "Stavros"
>>> "Hello, {}!".format(name)
Hello, Stavros!
>>> print(f"Hello, {name}!")
Hello, Stavros!
```

4. **Flow control statements:** Flow control statements are if, for, and while. There is no switch; instead, use if. Use for to enumerate through members of a list. To obtain a sequence of numbers you can iterate over, use range(<number>). These statements' syntax is thus:

```
>>> print(range(10))
range(0, 10)
>>> rangelist = list(range(10))
>>> print(rangelist)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

for number in range(10):
    # Check if number is one of
    # the numbers in the tuple.
    if number in (3, 4, 7, 9):
        # "Break" terminates a for without
        # executing the "else" clause.
        break
    else:
        # "Continue" starts the next iteration
        # of the loop. It's rather useless here,
        # as it's the last statement of the loop.
        continue
else:
    # The "else" clause is optional and is
    # executed only if the loop didn't "break".
    pass # Do nothing

if rangelist[1] == 2:
    print("The second item (lists are 0-based) is 2")
elif rangelist[1] == 3:
    print("The second item (lists are 0-based) is 3")
else:
    print("Dunno")

while rangelist[1] == 1:
    print("We are trapped in an infinite loop!")
```

5. **Functions:** Functions are declared with the def keyword. Optional arguments are set in the function declaration after the mandatory arguments by being assigned a default value. For named arguments, the name of the argument is assigned a value. Functions can return a tuple (and using tuple unpacking you can effectively return multiple values). Lambda functions are ad hoc functions that are comprised of a single statement. Parameters are passed by reference, but immutable types (tuples, ints, strings, etc) cannot be changed in the caller by the callee. This is

because only the memory location of the item is passed, and binding another object to a variable discards the old one, so immutable types are replaced. For example:

```
# Same as def funcvar(x): return x + 1
funcvar = lambda x: x + 1
>>> print(funcvar(1))
2

# an_int and a_string are optional, they have default values
# if one is not passed (2 and "A default string", respectively).
def passing_example(a_list, an_int=2, a_string="A default string"):
    a_list.append("A new item")
    an_int = 4
    return a_list, an_int, a_string

>>> my_list = [1, 2, 3]
>>> my_int = 10
>>> print(passing_example(my_list, my_int))
([1, 2, 3, 'A new item'], 4, "A default string")
>>> my_list
[1, 2, 3, 'A new item']
>>> my_int
10
```

6. **Classes:** Python supports a limited form of multiple inheritance in classes. Private variables and methods can be declared (by convention, this is not enforced by the language) by adding a leading underscore (e.g. _spam). We can also bind arbitrary names to class instances. An example follows:

```
class MyClass(object):
    common = 10
    def __init__(self):
        self.myvariable = 3
    def myfunction(self, arg1, arg2):
        return self.myvariable

    # This is the class instantiation

>>> classinstance = MyClass()
>>> classinstance.myfunction(1, 2)
3
# This variable is shared by all instances.
>>> classinstance2 = MyClass()
>>> classinstance.common
10
>>> classinstance2.common
10
# Note how we use the class name
# instead of the instance.
>>> MyClass.common = 30
>>> classinstance.common
30
>>> classinstance2.common
30
# This will not update the variable on the class,
# instead it will bind a new object to the old
# variable name.
>>> classinstance.common = 10
>>> classinstance.common
10
>>> classinstance2.common
30
>>> MyClass.common = 50
# This has not changed, because "common" is
# now an instance variable.
>>> classinstance.common
10
>>> classinstance2.common
50
```

```
# This class inherits from MyClass. The example
# class above inherits from "object", which makes
# it what's called a "new-style class".
# Multiple inheritance is declared as:
# class OtherClass(MyClass1, MyClass2, MyClassN)
class OtherClass(MyClass):
    # The "self" argument is passed automatically
    # and refers to the class instance, so you can set
    # instance variables as above, but from inside the class.
    def __init__(self, arg1):
        self.myvariable = 3
        print(arg1)

>>> classinstance = OtherClass("hello")
hello
>>> classinstance.myfunction(1, 2)
3
# This class doesn't have a .test member, but
# we can add one to the instance anyway. Note
# that this will only be a member of classinstance.
>>> classinstance.test = 10
>>> classinstance.test
10
```

7. **File I/O:** Python has a wide array of libraries built in. As an example, here is how serializing (converting data structures to strings using the pickle library) with file I/O is used:

```
import pickle
mylist = ["This", "is", 4, 13327]
# Open the file C:\\binary.dat for writing. The letter r before the
# filename string is used to prevent backslash escaping.
myfile = open(r"C:\\binary.dat", "wb")
pickle.dump(mylist, myfile)
myfile.close()

myfile = open(r"C:\\text.txt", "w")
myfile.write("This is a sample string")
myfile.close()

myfile = open(r"C:\\text.txt")
>>> print(myfile.read())
'This is a sample string'
myfile.close()

# Open the file for reading.
myfile = open(r"C:\\binary.dat", "rb")
loadedlist = pickle.load(myfile)
myfile.close()
>>> print(loadedlist)
['This', 'is', 4, 13327]
```

2) Solved Lab Activites

Sr.No	*Allocated Time*	*Level of Complexity*	*CLO Mapping*
1	*10*	*Low*	*CLO-2*
2	*10*	*Low*	*CLO-2*
3	*10*	*Low*	*CLO-2*
4	*10*	*Low*	*CLO-2*
5	*10*	*Low*	*CLO-2*
6	*10*	*Low*	*CLO-2*

Activity 1:

Python program to illustrate data types.

Solution:

```
>>> sample = [1, ["another", "list"], ("a", "tuple")]
>>> mylist = ["List item 1", 2, 3.14]
>>> mylist[0] = "List item 1 again" # We're changing the item.
>>> mylist[-1] = 3.21 # Here, we refer to the last item.
>>> mydict = {"Key 1": "Value 1", 2: 3, "pi": 3.14}
>>> mydict["pi"] = 3.15 # This is how you change dictionary values.
```

```
>>> mytuple = (1, 2, 3)
>>> myfunction = len
>>> print(myfunction(mylist))
```

Output

```
Value of myfunction:3
```

Activity 2:

Python program to illustrate indexing and slicing.

```
>>> mylist = ["List item 1", 2, 3.14]
>>> print(mylist[:])
['List item 1', 2, 3.1400000000000001]
>>> print(mylist[0:2])
['List item 1', 2]
>>> print(mylist[-3:-1])
['List item 1', 2]
>>> print(mylist[1:])
[2, 3.14]
# Adding a third parameter, "step" will have Python step in
# N item increments, rather than 1.
# E.g., this will return the first item, then go to the third and
# return that (so, items 0 and 2 in 0-indexing).
>>> print(mylist[::2])
['List item 1', 3.14]
```

Output

1. **Slicing the Entire List:**
   - `mylist[:]` returns all elements of `mylist`.
   - **Output:** `['List item 1', 2, 3.14]`
2. **Slicing from Index 0 to Index 2 (Exclusive):**
   - `mylist[0:2]` returns elements from the beginning up to, but not including, index 2.
   - **Output:** `['List item 1', 2]`
3. **Slicing with Negative Indices:**
   - `mylist[-3:-1]` returns elements from the third-to-last element up to, but not including, the last element.
   - **Output:** `['List item 1', 2]`
4. **Slicing from Index 1 to the End:**
   - `mylist[1:]` returns elements from index 1 to the end of the list.
   - **Output:** `[2, 3.14]`

5. **Slicing with Step Parameter:**
    - `mylist[::2]` returns every second element of the list, starting from the first element.
    - **Output:** `['List item 1', 3.14]`

Activity 3:

python program to illustrate flow control statement.

```
>>> print(range(10))
range(0, 10)
>>> rangelist = list(range(10))
>>> print(rangelist)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

for number in range(10):
    # Check if number is one of
    # the numbers in the tuple.
    if number in (3, 4, 7, 9):
        # "Break" terminates a for without
        # executing the "else" clause.
        break
    else:
        # "Continue" starts the next iteration
        # of the loop. It's rather useless here,
        # as it's the last statement of the loop.
        continue
else:
    # The "else" clause is optional and is
    # executed only if the loop didn't "break".
    pass # Do nothing

if rangelist[1] == 2:
    print("The second item (lists are 0-based) is 2")
elif rangelist[1] == 3:
    print("The second item (lists are 0-based) is 3")
else:
    print("Dunno")

while rangelist[1] == 1:
    print("We are trapped in an infinite loop!")
```

Output

1. range(0, 10)
2. [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
3. Dunno

4. We are trapped in an infinite loop!
5. We are trapped in an infinite loop!
6. We are trapped in an infinite loop! ...

Activity 4:

python program to illustrate function.

```
# Same as def funcvar(x): return x + 1
funcvar = lambda x: x + 1
>>> print(funcvar(1))
2

# an_int and a_string are optional, they have default values
# if one is not passed (2 and "A default string", respectively).
def passing_example(a_list, an_int=2, a_string="A default string"):
    a_list.append("A new item")
    an_int = 4
    return a_list, an_int, a_string

>>> my_list = [1, 2, 3]
>>> my_int = 10
>>> print(passing_example(my_list, my_int))
([1, 2, 3, 'A new item'], 4, "A default string")
>>> my_list
[1, 2, 3, 'A new item']
>>> my_int
10
```

Output

```
Shown above
```

Activity 5:

python program to illustrate class.

```
class MyClass(object):
    common = 10
    def __init__(self):
        self.myvariable = 3
    def myfunction(self, arg1, arg2):
        return self.myvariable

    # This is the class instantiation

>>> classinstance = MyClass()
```

```
>>> classinstance.myfunction(1, 2)
3
# This variable is shared by all instances.
>>> classinstance2 = MyClass()
>>> classinstance.common
10
>>> classinstance2.common
10
# Note how we use the class name
# instead of the instance.
>>> MyClass.common = 30
>>> classinstance.common
30
>>> classinstance2.common
30
# This will not update the variable on the class,
# instead it will bind a new object to the old
# variable name.
>>> classinstance.common = 10
>>> classinstance.common
10
>>> classinstance2.common
30
>>> MyClass.common = 50
# This has not changed, because "common" is
# now an instance variable.
>>> classinstance.common
10
>>> classinstance2.common
50

# This class inherits from MyClass. The example
# class above inherits from "object", which makes
# it what's called a "new-style class".
# Multiple inheritance is declared as:
# class OtherClass(MyClass1, MyClass2, MyClassN)
class OtherClass(MyClass):
    # The "self" argument is passed automatically
    # and refers to the class instance, so you can set
    # instance variables as above, but from inside the class.
    def __init__(self, arg1):
        self.myvariable = 3
        print(arg1)

>>> classinstance = OtherClass("hello")
hello
>>> classinstance.myfunction(1, 2)
```

```
3
# This class doesn't have a .test member, but
# we can add one to the instance anyway. Note
# that this will only be a member of classinstance.
>>> classinstance.test = 10
>>> classinstance.test
10
```

Output

Shown above

Activity 6:

python program to illustrate IO handling.

```
import pickle
mylist = ["This", "is", 4, 13327]
# Open the file C:\\binary.dat for writing. The letter r before the
# filename string is used to prevent backslash escaping.
myfile = open(r"C:\\binary.dat", "wb")
pickle.dump(mylist, myfile)
myfile.close()

myfile = open(r"C:\\text.txt", "w")
myfile.write("This is a sample string")
myfile.close()

myfile = open(r"C:\\text.txt")
>>> print(myfile.read())
'This is a sample string'
myfile.close()

# Open the file for reading.
myfile = open(r"C:\\binary.dat", "rb")
loadedlist = pickle.load(myfile)
myfile.close()
>>> print(loadedlist)
['This', 'is', 4, 13327]
```

Output

Shown above

3) Graded Lab Tasks

The purpose of this lab task is to help students understand and manipulate different data structures in Python, specifically lists, dictionaries, and tuples. Students will perform various operations on these data structures, including creation, modification, and data access.

Lab Task 1

Instructions:

1. **List Operations:**
    - Create a list with the following elements: `["red", "green", "blue", "yellow", "purple"]`.
    - Add `"orange"` to the end of the list.
    - Insert `"pink"` at the second position.
    - Remove `"blue"` from the list.
    - Reverse the list.
    - Sort the list in alphabetical order.
    - Slice the list to get the last three elements.
    - Print the length of the list.
2. **Dictionary Operations:**
    - Create a dictionary with the following key-value pairs: `{"name": "John", "age": 25, "city": "New York"}`.
    - Add a new key-value pair for `"email": "john.doe@example.com"`.
    - Update the `"age"` to 26.
    - Remove the `"city"` key from the dictionary.
    - Check if the key `"name"` exists in the dictionary.
    - Print all the keys and values of the dictionary.
3. **Tuple Operations:**
    - Create a tuple with the following elements: `(10, 20, 30, 40, 50)`.
    - Access and print the third element of the tuple.
    - Try to change the second element of the tuple and observe the result (Note: Tuples are immutable).
    - Create a new tuple by concatenating the existing tuple with `(60, 70, 80)`.
    - Unpack the tuple into individual variables and print them.
4. **Combining Data Structures:**
    - Create a dictionary where the keys are strings and the values are lists. For example: `{"fruits": ["apple", "banana", "cherry"], "vegetables": ["carrot", "broccoli", "spinach"]}`.
    - Add a new fruit `"orange"` to the list of fruits.
    - Remove `"spinach"` from the list of vegetables.
    - Print the updated dictionary.
    - Create a list of tuples, where each tuple contains a fruit and its color, e.g., `[("apple", "red"), ("banana", "yellow"), ("cherry", "red")]`.
    - Access and print the color of `"banana"` from the list of tuples.

Submission:

1. Create a Python script file (e.g., `lab_data_structures.py`) containing all the operations mentioned above.
2. Add comments in your script explaining each step.
3. Test your script to ensure all tasks are performed correctly.
4. Submit the Python script file along with a brief report (1-2 pages) explaining your observations and any challenges you faced during the lab.

Lab Task 2

The purpose of this lab task is to help students understand how to define and use functions and classes in Python. Students will create functions for specific tasks and design classes that encapsulate data and behavior.

Instructions:

1. **Function Definition and Usage:**
    - Write a function `greet` that takes a name as an argument and prints a greeting message.
    - Write a function `fibonacci` that takes an integer `n` and returns the `n`th Fibonacci number.
    - Write a function `is_prime` that takes an integer and returns `True` if the number is prime, and `False` otherwise.
    - Write a function `factorial` that takes an integer `n` and returns the factorial of `n`.
    - Write a function `reverse_string` that takes a string and returns the reversed string.
2. **Class Definition and Usage:**
    - Define a class `Person` with the following attributes: `name`, `age`, and `email`.
    - Add a method `introduce` to the `Person` class that prints an introduction message using the person's attributes.
    - Define a class `Student` that inherits from `Person` and adds an attribute `student_id`.
    - Add a method `study` to the `Student` class that prints a message indicating the student is studying.
    - Define a class `Course` with attributes `course_name`, `students` (a list of `Student` objects), and a method `add_student` to add a student to the course.
    - Add a method `list_students` to the `Course` class that prints the names of all students enrolled in the course.
3. **Combining Functions and Classes:**
    - Create an instance of the `Course` class with the course name `"Introduction to Python"`.
    - Create three instances of the `Student` class with different names, ages, emails, and student IDs.
    - Add the students to the course using the `add_student` method.
    - List all students in the course using the `list_students` method.
    - Use the `introduce` method to print introduction messages for each student.
    - Demonstrate the `study` method for one of the students.

Submission:

1. Create a Python script file (e.g., `lab_functions_classes.py`) containing all the operations mentioned above.
2. Add comments in your script explaining each step.
3. Test your script to ensure all tasks are performed correctly.
4. Submit the Python script file along with a brief report (1-2 pages) explaining your observations and any challenges you faced during the lab.

Lab Task 3

`The purpose of this lab task is to help students understand how to perform input/output (I/O) operations using the pickle module in Python. Students will learn how to serialize (pickle) and deserialize (unpickle) Python objects and store them in files.`

Instructions:

1. **Understanding Pickling:**
   - Write a brief explanation (in comments or a separate markdown file) of what pickling is and why it is used in Python.
2. **Basic Pickling and Unpickling:**
   - Create a list of dictionaries where each dictionary represents a student with keys `name`, `age`, and `grade`. For example:

     ```
     python
     Copy code
     students = [
         {"name": "Alice", "age": 20, "grade": "A"},
         {"name": "Bob", "age": 22, "grade": "B"},
         {"name": "Charlie", "age": 19, "grade": "C"}
     ]
     ```

   - Use the `pickle` module to serialize the list of students and save it to a file called `students.pkl`.
   - Write a function `load_students` that deserializes the data from `students.pkl` and returns the list of students.
   - Print the loaded students to verify the data.
3. **Working with Custom Classes:**
   - Define a class `Student` with attributes `name`, `age`, and `grade`.
   - Modify the previous list of dictionaries to a list of `Student` objects.
   - Serialize the list of `Student` objects and save it to a file called `students_class.pkl`.
   - Write a function `load_student_objects` that deserializes the data from `students_class.pkl` and returns the list of `Student` objects.
   - Print the loaded student objects to verify the data.
4. **Advanced Operations:**
   - Write a function `add_student` that takes a file name, a `Student` object, and adds the student to the list in the file.

- Write a function `remove_student` that takes a file name and a student name, and removes the student with that name from the file.
- Demonstrate the usage of `add_student` and `remove_student` functions by adding a new student and removing an existing student from `students_class.pkl`.

Submission:

1. Create a Python script file (e.g., `lab_pickle_io.py`) containing all the operations mentioned above.
2. Add comments in your script explaining each step.
3. Test your script to ensure all tasks are performed correctly.
4. Submit the Python script file along with a brief report (1-2 pages) explaining your observations and any challenges you faced during the lab.